

Autonomous Claims Orchestrator - Comprehensive Backend Workflow Documentation

Scope

- Coverage: complete backend workflow, all backend Python files, backend dependency manifest, and all data files under data/ that contribute to runtime.
- Total backend Python files documented: **30**.
- Additional backend dependency file documented: **backend/requirements.txt**.
- Total microservices (folder-level business services): **8** -> claim_notification, dashboard, decision, email_ingestion, extraction, faq_resolution, ingested_claims, process_claim.
- Additional runtime modules: API gateway (backend/fastapi_server.py) and shared config (backend/common/config.py).

End-to-End Workflow

1. **API endpoint**: FastAPI routes in backend/fastapi_server.py receive sync/process/read/clear requests.
2. **Inbox sync + classification**: backend/email_ingestion/service.py pulls IMAP emails, filters non-FNOL, routes FAQ mail, and forwards valid claims to ingestion storage.
3. **FAQ branch**: backend/faq_resolution/service.py answers FAQ/customer-data questions and stores conversation traces via ingested-claims service.
4. **Ingested claim persistence**: backend/ingested_claims/service.py stores claim records and attachments, performs dedup/thread merge, and constructs mailChain using mail_chain.py.
5. **Orchestration**: backend/process_claim/orchestrator.py loads ingested claim -> runs extraction -> builds decision pack -> persists processed claim.
6. **Extraction**: backend/extraction/service.py extracts structured data from email + attachments using text/vision models.
7. **Decisioning**: backend/decision/service.py combines extracted signals with policy grounding from local policy data and clause matching logic.
8. **Processed claim persistence + KPIs**: backend/dashboard/service.py stores processed outputs and computes dashboard KPIs/CSV history.
9. **Notification side effects**: backend/claim_notification/service.py sends FNOL acknowledgement, under-review updates, and desk rejection emails.

Microservice Count

- **8 microservices**: backend/claim_notification, backend/dashboard, backend/decision, backend/email_ingestion, backend/extraction, backend/faq_resolution, backend/ingested_claims, backend/process_claim
- With gateway + shared runtime modules included, total participating backend modules = **10**.

Backend Dependency File

- backend/requirements.txt: Python package dependency list required to run backend services and integrations (FastAPI, OpenAI client, etc.).

File-by-File Sequential Technical Definition

`backend/common/config.py`

- **Purpose**: Shared configuration for backend services.
- **Outbound references (imports/calls)**: None
- **Inbound references (imported by)**: backend.claim_notification.service, backend.dashboard.service, backend.email_ingestion.service, backend.extraction.service, backend.faq_resolution.service, backend.ingested_claims.service
- **Functions**:

- `_get_path_from_env()`: Get path from environment variable or use fallback.
- `_get_project_root()`: Get project root directory.
- `ensure_data_dir()`: Ensure all required data directories exist.
- `get_config_summary()`: Get a summary of current configuration paths.

``backend/fastapi_server.py``

- **Purpose**: FastAPI Server for Autonomous Claims Orchestrator.
- **Outbound references (imports/calls)**: `backend.dashboard.service`, `backend.email_ingestion.service`, `backend.ingested_claims.service`, `backend.process_claim.orchestrator`
- **Inbound references (imported by)**: None
- **Classes**:
 - `ProcessClaimRequest`: Helper/handler that `ProcessClaimRequest`.
 - `SaveClaimRequest`: Helper/handler that `SaveClaimRequest`.
 - `SyncInboxResponse`: Helper/handler that `SyncInboxResponse`.
- **Functions**:
 - `health_check()`: Health check endpoint.
 - `process_claim_endpoint()`: Process an ingested claim end-to-end.
 - `get_ingested_claims()`: Get list of ingested claims.
 - `get_ingested_claim_by_id_endpoint()`: Get a specific ingested claim by ID.
 - `clear_ingested_claims()`: Clear all ingested claims.
 - `clear_processed_claims()`: Clear processed-claims index, CSV, and all stored processed claim JSON files.
 - `get_claims_list()`: Get list of processed claim summaries.
 - `save_claim()`: Save a processed claim.
 - `get_claim_by_id()`: Get a processed claim by ID.
 - `get_dashboard_kpis_endpoint()`: Get dashboard KPIs and statistics.
 - `sync_inbox_endpoint()`: Sync email inbox and ingest new claims.
 - `root()`: Root endpoint with API information.

``backend/__init__.py``

- **Purpose**: Autonomous Claims Orchestrator - Backend Services.
- **Outbound references (imports/calls)**: None
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

``backend/claim_notification/__init__.py``

- **Purpose**: Claim notification service.
- **Outbound references (imports/calls)**: `backend.claim_notification.service`
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

``backend/claim_notification/service.py``

- **Purpose**: Claim Notification Service.
- **Outbound references (imports/calls)**: `backend.common.config`
- **Inbound references (imported by)**: `backend.claim_notification.__init__`, `backend.dashboard.service`, `backend.ingested_claims.service`, `backend.process_claim.orchestrator`

- **Functions**:

- `_load_env()`: Load env vars from `.env` if present.
- `_extract_email_address()`: Extract plain email from 'Name ' or return as-is if already plain.
- `_claimant_reply_to_email()`: Prefer original From; fall back to extracted contact email from the draft.
- `_send_smtp_alternative()`: Helper/handler that send smtp alternative.
- `send_fnol_received_acknowledgement_email()`: Send immediate acknowledgement when a new FNOL email is ingested (before human/AI processing).
- `send_claim_under_review_email()`: Send a professional "Your claim is being reviewed" email to the claimant.
- `send_desk_rejection_email()`: Send a professional desk rejection email when a claim is auto-rejected.

`backend/common/__init__.py`

- **Purpose**: Shared utilities for backend services.
- **Outbound references (imports/calls)**: None
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

`backend/dashboard/__init__.py`

- **Purpose**: Dashboard Microservice.
- **Outbound references (imports/calls)**: `backend.dashboard.service`
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

`backend/dashboard/__main__.py`

- **Purpose**: CLI entry for dashboard microservice.
- **Outbound references (imports/calls)**: `backend.dashboard.service`
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

`backend/dashboard/service.py`

- **Purpose**: Dashboard Microservice - Processed Claims History.
- **Outbound references (imports/calls)**: `backend.claim_notification.service`, `backend.common.config`
- **Inbound references (imported by)**: `backend.dashboard.__init__`, `backend.dashboard.__main__`, `backend.fastapi_server`, `backend.process_claim.orchestrator`
- **Functions**:
 - `_get_index()`: Load claims index.
 - `_save_index()`: Save claims index.
 - `_escape_csv()`: Escape value for CSV.
 - `_append_to_csv()`: Append claim row to CSV.
 - `save_processed_claim()`: Save a processed claim to history and CSV.
 - `get_processed_claim_summaries()`: Get list of processed claim summaries.
 - `get_processed_claim_by_id()`: Get full claim data by ID.
 - `clear_all_processed_claims()`: Remove processed-claims index, CSV, and per-claim JSON files.
 - `get_csv_content()`: Get CSV content for export.
 - `get_dashboard_kpis()`: Compute real-time KPIs from all processed claims.
 - `main()`: CLI entry point.

``backend/decision/__init__.py``

- **Purpose**: Decision Microservice.
- **Outbound references (imports/calls)**: backend.decision.service
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

``backend/decision/policy_clauses.py``

- **Purpose**: Policy Clauses Module.
- **Outbound references (imports/calls)**: None
- **Inbound references (imported by)**: backend.decision.service
- **Functions**:
 - `_infer_product_types()`: Infer product types from policy number prefix.
 - `_infer_loss_types()`: Infer loss types from claim text.
 - `_compute_similarity()`: Compute similarity score for clause matching.
 - `get_policy_grounding()`: Get policy grounding (matching clauses) based on extracted claim fields.

``backend/decision/policy_clauses_realistic.py``

- **Purpose**: Realistic Policy Clauses Database.
- **Outbound references (imports/calls)**: None
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

``backend/decision/policy_grounding_local.py``

- **Purpose**: Policy Grounding using Local JSON Database Files.
- **Outbound references (imports/calls)**: None
- **Inbound references (imported by)**: backend.decision.service, backend.faq_resolution.service
- **Functions**:
 - `_load_data()`: Load data from JSON files into cache.
 - `find_customer_by_policy()`: Find customer by policy number.
 - `find_customer_by_email()`: Find customer by email (e.g. from inbound email sender).
 - `get_policies_for_customer()`: Get all policies for a customer (for FAQ lookups by email).
 - `get_policy_by_number()`: Get policy by policy number.
 - `get_policy_details_by_policy_number()`: Get all policy details for a specific policy number.
 - `_calculate_confidence_score()`: Calculate confidence score following mapping JSON formula.
 - `_get_recommendation()`: Get recommendation based on confidence score thresholds.
 - `get_policy_grounding_from_local_data()`: Get policy grounding from local JSON data files following mapping JSON workflow.
 - `get_complete_policy_info()`: Get complete policy information including customer, policy, and all details.

``backend/decision/service.py``

- **Purpose**: Decision Service.
- **Outbound references (imports/calls)**: backend.decision.policy_clauses, backend.decision.policy_grounding_local
- **Inbound references (imported by)**: backend.decision.__init__, backend.process_claim.orchestrator
- **Functions**:
 - `_mask_email()`: Mask email for PII protection.

- `_mask_phone()`: Mask phone for PII protection.
- `_build_claim_draft()`: Build claim draft from extraction result.
- `_iso_now()`: Current time as ISO string.
- `build_decision_pack()`: Build full decision pack and claim data.

``backend/email_ingestion/___init___py``

- **Purpose**: Email Ingestion Microservice.
- **Outbound references (imports/calls)**: `backend.email_ingestion.service`
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

``backend/email_ingestion/___main___py``

- **Purpose**: CLI entry for email_ingestion microservice.
- **Outbound references (imports/calls)**: `backend.email_ingestion.service`
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

``backend/email_ingestion/service.py``

- **Purpose**: Email Ingestion Microservice.
- **Outbound references (imports/calls)**: `backend.common.config`, `backend.faq_resolution.service`, `backend.ingested_claims.service`
- **Inbound references (imported by)**: `backend.email_ingestion.__init__`, `backend.email_ingestion.__main__`, `backend.fastapi_server`, `backend.ingested_claims.__main__`
- **Functions**:
 - `_norm_key()`: Lowercase + strip for dedup key comparison (mirrors `ingested_claims._normalize_key`).
 - `_strip_angle()`: Strip angle brackets from a Message-ID (mirrors `ingested_claims._inner_message_id`).
 - `_load_env()`: Load env vars from `.env` if present.
 - `_text()`: Helper/handler that text.
 - `_primary_message_text()`: Text the sender most likely authored (above quoted reply / forward).
 - `_has_insurance_or_claim_context()`: Signals that the email is about an insurance / policy claim, not a generic product complaint.
 - `_has_explicit_claim_phrasing()`: Clear intent to file or discuss an insurance claim / FNOL (passes without peril words).
 - `_has_peril_language()`: Damage / loss / accident wording (too broad alone — needs insurance context).
 - `_keyword_surface_text()`: Subject + latest non-quoted segment so reply-all threads do not inherit quoted keywords.
 - `_has_strong_insurance_fnol_signal()`: Clear P&C; / FNOL signals in the surface text (not consumer warranty templates).
 - `_thread_smells_like_oem_customer_complaint()`: Retail / manufacturer complaint workflows (RMA, product complaint, doc requests).
 - `_should_reject_consumer_complaint_correspondence()`: Reject retail / customer-support complaint threads and document follow-ups unless.
 - `_has_relevant_keywords()`: Gate for FNOL ingestion: insurance-claim intent, not generic complaints.
 - `_has_strong_keywords()`: Strong FNOL indicators for LLM-offline fallback: explicit claim/FNOL or peril + insurance context.
 - `should_ingest_incoming_email()`: Public entry: same classification as IMAP sync (keywords + optional LLM).
 - `_classify_fnol_by_llm()`: LLM-based FNOL classifier — keywords on latest message; LLM judges FNOL vs complaint follow-up.
 - `_strip_html()`: Strip HTML tags for plain text body.
 - `_get_part_text()`: Extract plain text from MIME part.
 - `_decode_header_value()`: Decode MIME-encoded header.

- `_format_address()`: Format email address for display.
- `_build_full_email_body()`: Build full email string with headers.
- `_extract_raw_message()`: Extract raw RFC822 message bytes from IMAP fetch response.
- `_extract_body_text()`: Extract plain text body from email - ONLY the actual email body, NOT attachments or forwarded content.
- `extract_plain_body_from_rfc822()`: Parse raw MIME bytes and return plain text (SendGrid when text/html fields are empty).
- `sync_inbox()`: Connect to IMAP, fetch emails, filter by FNOL, save claims.
- `main()`: CLI entry point.

``backend/extraction/__init__.py``

- **Purpose**: Extraction Microservice.
- **Outbound references (imports/calls)**: `backend.extraction.service`
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

``backend/extraction/__main__.py``

- **Purpose**: CLI entry for extraction microservice.
- **Outbound references (imports/calls)**: `backend.extraction.service`
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

``backend/extraction/service.py``

- **Purpose**: Extraction Microservice.
- **Outbound references (imports/calls)**: `backend.common.config`
- **Inbound references (imported by)**: `backend.extraction.__init__`, `backend.extraction.__main__`, `backend.process_claim.orchestrator`
- **Functions**:
 - `_load_env()`: Load environment variables from `.env` if present.
 - `_get_openai_client()`: Get OpenAI client (lazy import).
 - `_is_image_file()`: Check if file is an image suitable for Vision API.
 - `_read_text_file()`: Read text file content safely.
 - `extract_from_email()`: Extract structured claim fields from email body using OpenAI LLM.
 - `_extract_schemas()`: Return document extraction schemas for LLM prompt.
 - `extract_from_document()`: Classify document and extract key fields using LLM.
 - `analyze_image_with_vision()`: Analyze image using OpenAI Vision API. Returns structured KPIs and detailed summary.
 - `extract_claim_information()`: Main entry: extract information from email and all attachments.
 - `main()`: CLI entry: run extraction for a claim and output JSON.

``backend/faq_resolution/__init__.py``

- **Purpose**: FAQ Auto-Resolution Microservice.
- **Outbound references (imports/calls)**: None
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

``backend/faq_resolution/__main__.py``

- **Purpose**: CLI entry for FAQ resolution microservice.

- **Outbound references (imports/calls)**: backend.faq_resolution.service
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

`backend/faq_resolution/service.py`

- **Purpose**: FAQ Auto-Resolution Service.
- **Outbound references (imports/calls)**: backend.common.config, backend.decision.policy_grounding_local
- **Inbound references (imported by)**: backend.email_ingestion.service, backend.faq_resolution.__main__
- **Functions**:
 - `_load_env()`: Load env vars from .env if present.
 - `_load_faq_data()`: Load FAQ data from CSV file.
 - `_is_faq_query()`: Use LangChain agent to detect if email is an FAQ query.
 - `_classify_customer_data_intent()`: Classify if the question is a customer-data query: policy_number, policy_expiry,.
 - `_extract_policy_number_from_body()`: Extract policy number from email body if customer mentioned it. Tolerates typo 'poicy'.
 - `_answer_customer_data_query()`: Answer customer-data FAQ by querying the policy DB.
 - `_find_faq_answer()`: Find the best matching FAQ answer from FAQ.csv (semantic/LLM or keyword fallback).
 - `_send_faq_response_email()`: Send FAQ response email using SMTP.
 - `process_faq_email()`: Main entry point for FAQ processing.

`backend/ingested_claims/__init__.py`

- **Purpose**: Ingested Claims Microservice.
- **Outbound references (imports/calls)**: backend.ingested_claims.service
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

`backend/ingested_claims/__main__.py`

- **Purpose**: CLI entry for ingested_claims microservice.
- **Outbound references (imports/calls)**: backend.email_ingestion.service, backend.ingested_claims.service
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

`backend/ingested_claims/mail_chain.py`

- **Purpose**: Build a chronological mail chain (oldest → newest) for inbox UI.
- **Outbound references (imports/calls)**: None
- **Inbound references (imported by)**: backend.ingested_claims.service
- **Functions**:
 - `_strip_html()`: Helper/handler that strip html.
 - `_html_to_plain_preserve_breaks()`: Strip tags but keep line breaks so Gmail/Outlook reply delimiters can match.
 - `_decode_header_value()`: Helper/handler that decode header value.
 - `_format_address()`: Helper/handler that format address.
 - `_get_part_text()`: Helper/handler that get part text.
 - `_body_from_simple_part()`: Helper/handler that body from simple part.
 - `_extract_from_multipart_alternative()`: Helper/handler that extract from multipart alternative.
 - `_extract_body_skip_rfc822()`: Main visible body of this message, ignoring nested message/rfc822 parts.

- `_count_attachments()`: Helper/handler that count attachments.
- `_parse_date_header()`: Helper/handler that parse date header.
- `_message_to_entry()`: Helper/handler that message to entry.
- `_inner_from_rfc822_part()`: Helper/handler that inner from rfc822 part.
- `_collect_nested_rfc822_messages()`: Deduplicated inner messages from message/rfc822 parts, in walk (document) order.
- `_body_suggests_reply_thread()`: Heuristic: stored body may contain quoted history worth re-splitting.
- `_split_reply_thread()`: Split a plain-text body into segments (newest first), using common client delimiters.
- `_parse_outlook_headers_from_segment()`: If segment starts with From:/Sent:/To:/Subject:/Date: lines, parse them; return meta + body.
- `_entries_from_plain_thread()`: Helper/handler that entries from plain thread.
- `build_mail_chain()`: Build mailChain list (oldest → newest). Prefer MIME structure when raw bytes are available.
- `ensure_mail_chain_on_claim()`: Mutate claim to always include mailChain; rebuild single-segment rows when body looks like a thread.

``backend/ingested_claims/service.py``

- **Purpose**: Ingested Claims Service.
- **Outbound references (imports/calls)**: `backend.claim_notification.service`, `backend.common.config`, `backend.ingested_claims.mail_chain`
- **Inbound references (imported by)**: `backend.email_ingestion.service`, `backend.fastapi_server`, `backend.ingested_claims.__init__`, `backend.ingested_claims.__main__`, `backend.process_claim.orchestrator`
- **Functions**:
 - `_normalize_key()`: Normalize string for deduplication.
 - `_normalized_subject_from()`: Build normalized dedup key from subject and from.
 - `_parseaddr_email()`: Helper/handler that parseaddr email.
 - `normalize_thread_subject()`: Strip reply/forward prefixes so 'Re: FNOL' matches the original subject.
 - `_inner_message_id()`: Helper/handler that inner message id.
 - `_message_ids_from_header_value()`: Extract Message-ID tokens from In-Reply-To / References (angle brackets optional).
 - `_all_message_id_tokens_for_claim()`: Helper/handler that all message id tokens for claim.
 - `_fallback_policy_display()`: Use email Message-ID as identifier when policy number not found.
 - `extract_policy_number()`: Extract policy number from email body using common patterns.
 - `_get_faq_answered_ids()`: Get dedup keys for emails we already answered via FAQ (not ingested as claims).
 - `add_faq_answered_id()`: Record that we answered this email as FAQ so we don't re-answer or ingest it.
 - `get_existing_message_ids()`: Build the dedup key set used to skip already-processed emails in sync.
 - `get_faq_answered_id_set()`: Return the set of IDs for emails already answered as FAQ (used to skip re-sending replies).
 - `is_reply_to_existing_claim()`: Return True if this email looks like a follow-up / reply to an already-ingested claim.
 - `add_dedup_keys_to_set()`: Add dedup keys for a newly ingested / merged email so it isn't re-processed in the same sync run.
 - `is_duplicate_email()`: True only if this exact message was already ingested or merged.
 - `_message_id_already_ingested()`: Helper/handler that message id already ingested.
 - `_find_claim_by_rfc_message_id()`: Find claim whose messageId / threadMessageIds matches this RFC Message-ID.
 - `_find_claim_for_thread_merge()`: Locate existing claim for a follow-up (In-Reply-To / References, else normalized subject + sender).
 - `_merge_follow_up_into_claim()`: Append follow-up content to an existing claim (mail chain + body + attachments + thread ids).
 - `_find_existing_duplicate_claim()`: Return an existing ingested claim row if this message matches, else None.
 - `_get_claims_data()`: Load ingested claims from JSON.
 - `_save_claims_data()`: Save ingested claims to JSON.

- `_seed_demo_claims()`: Seed demo claims from demo-data/scenarios.
- `_iso_now()`: Convert milliseconds to ISO string.
- `_parse_mail_date_header()`: Return (iso_utc_or_empty, human_display_or_empty) from RFC 2822 Date header.
- `save_ingested_claim()`: Save ingested claim to JSON and attachments to disk.
- `append_outbound_mail_entry()`: Append an outbound (system-sent) email as a mail-chain entry to an existing claim.
- `save_faq_conversation()`: Create a claim record (source='imap_faq') that holds both the customer's FAQ email and.
- `get_all_ingested_claims()`: Get all ingested claims. Seeds demo only if no real emails exist.
- `get_ingested_claim_by_id()`: Get a single claim by ID.
- `get_policy_numbers()`: Get policy numbers for dropdown. Only shows demo if no real emails exist.
- `clear_all_ingested_claims()`: Clear all ingested claims.
- `read_attachment_content()`: Read attachment content for processing.

``backend/process_claim/__init__.py``

- **Purpose**: Process Claim Orchestrator.
- **Outbound references (imports/calls)**: `backend.process_claim.orchestrator`
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

``backend/process_claim/__main__.py``

- **Purpose**: CLI entry for process_claim orchestrator.
- **Outbound references (imports/calls)**: `backend.process_claim.orchestrator`
- **Inbound references (imported by)**: None
- **Functions**: None (package marker/export module).

``backend/process_claim/orchestrator.py``

- **Purpose**: Process Claim Orchestrator.
- **Outbound references (imports/calls)**: `backend.claim_notification.service`, `backend.dashboard.service`, `backend.decision.service`, `backend.extraction.service`, `backend.ingested_claims.service`
- **Inbound references (imported by)**: `backend.fastapi_server`, `backend.process_claim.__init__`, `backend.process_claim.__main__`
- **Functions**:
 - `_should_auto_reject_for_expired_policy()`: Determine if the claim should be auto-rejected due to expired or invalid policy.
 - `process_claim()`: Process an ingested claim end-to-end.
 - `main()`: CLI entry: process a claim and output JSON.

Data Files Contributing to Workflow

- `data/FAQ.csv`: FAQ knowledge base read by FAQ resolution service.
- `data/drafts/DRAFT-CLM-ING-1769833440467-ro1vdnc-1770053921268.json`: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- `data/drafts/DRAFT-CLM-ING-1769833441648-7bgo9f3-1770313590960.json`: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- `data/drafts/DRAFT-CLM-ING-1769838030177-9yijn6x-1769877501097.json`: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- `data/drafts/DRAFT-CLM-ING-1769838030177-9yijn6x-176987774253.json`: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- `data/drafts/DRAFT-CLM-ING-1769838030177-9yijn6x-1769923879184.json`: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.

- data/drafts/DRAFT-CLM-ING-1769838030177-9yijn6x-1769936376150.json: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/drafts/drafts-index.json: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/faq-answered-ids.json: FAQ dedup state; read/write by ingested claims service.
- data/ingested-attachments/ING-1774850988061-b03e970/black-car-damaged-by-a-road-accident-photo.jpg: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/ingested-attachments/ING-1774850988061-b03e970/damaged_car.jpeg: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/ingested-attachments/ING-1774850988061-b03e970/police-report.txt: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/ingested-attachments/ING-1774850988061-b03e970/repair-estimate.txt: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/ingested-attachments/ING-1774850996166-de0b91b/emergency-invoice.txt: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/ingested-attachments/ING-1774850996166-de0b91b/roof-leaking-water-damage.jpg: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/ingested-attachments/ING-1774850996166-de0b91b/water-extraction-estimate.txt: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/ingested-attachments/ING-1774851070414-c33e073/emergency-invoice.txt: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/ingested-attachments/ING-1774851070414-c33e073/roof-leaking-water-damage.jpg: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/ingested-attachments/ING-1774851070414-c33e073/water-extraction-estimate.txt: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/ingested-attachments/ING-1774851079889-67d2d74/damaged_car.jpeg: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/ingested-attachments/ING-1774851079889-67d2d74/police-report.txt: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/ingested-attachments/ING-1774851079889-67d2d74/repair-estimate.txt: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/ingested-claims.json: Canonical ingested claim store; read/write by ingested claims service.
- data/processed-claims/CLM-ING-1774850988061-b03e970.json: Runtime artifact/sample payload under data storage; contributes as persisted input/output context.
- data/processed-claims/claims-history.csv: Append-only processed claims history; maintained by dashboard service.
- data/processed-claims/claims-index.json: Processed claim index; read/write by dashboard service.

Cross-Reference Summary

- API gateway cross-references ingestion, process orchestration, and dashboard modules.
- Orchestrator cross-references extraction + decision + dashboard + notification services.
- FAQ and ingestion services cross-reference each other for diversion and persisted conversation history.
- Decision service cross-references local policy grounding and clause fallback modules.
- Dashboard and ingested-claims services cross-reference notification service for outbound customer communication.